

Progress Report: Quantum++ Simulations for HHL and Nozzle-Flow Mean Estimation

Zhiwei

May 25, 2026

Abstract

This project explores two quantum-algorithm examples using the Quantum++ C++ library: a Harrow-Hassidim-Lloyd (HHL) linear-system solver and a Quantum-Amplitude-Estimation-Algorithm (QAEA) backend for a quasi-one-dimensional nozzle flow problem. The current implementation is intended as a simulation and learning platform rather than a performance-competitive solver. This report summarizes the code structure, current implementation status, relation to the reference nozzle-flow paper, known limitations, and possible directions for further development.

1 Project Goals

The project has two related goals. The first is to implement and understand the main steps of the HHL algorithm for solving small linear systems in a simulated quantum-circuit setting. The second is to investigate how quantum mean estimation, specifically a QAEA-style subroutine, can be connected to a quasi-one-dimensional nozzle-flow ODE solver inspired by the paper *Finding flows of a Navier-Stokes fluid through quantum computing*.

The current emphasis is on correctness of structure, code clarity, and comparison between classical and quantum-simulated components. Since the code uses Quantum++ as a classical state-vector simulator, the present work does not claim quantum speedup.

2 Code Organization

The repository contains two main executable examples.

- `HHL.cpp`: Runs the HHL linear-system demo.
- `nozzle.cpp`: Runs the nozzle-flow demo with both classical and QAEA-style mean-estimation backends.

The HHL-related code is organized into:

- `include/HHLinput.hpp` and `source/HHLinput.cpp`: input parsing, Hermitian validation, matrix scaling, vector normalization, and sparsity estimation.
- `include/HHLSolver.hpp` and `source/HHLSolver.cpp`: the simulated HHL circuit, including quantum phase estimation, conditional ancilla rotation, inverse phase estimation, and postselection.

The nozzle-related code is organized into:

- `include/nozzle_qns/nozzle_types.hpp`: common state and gas model types.
- `include/nozzle_qns/nozzle_geometry.hpp`: one-dimensional grid and nozzle area function.
- `quasild_driver`: conversion between conservative and primitive variables, inviscid fluxes, source terms, and spatial derivatives.
- `boundary_conditions`: inlet and outlet boundary conditions.
- `steady_state_ic`: initial-condition construction based on a perturbed steady-state nozzle solution.
- `interval_stepper` and `quantum_ode_solver`: time stepping and ODE orchestration.
- `mean_estimator`, `classical_mean_estimator`, and `qpp_mean_estimator`: the backend interface and its classical and Quantum++ QAEA-style implementations.

The HTML documentation provides class, file, and function navigation while excludes vendored third-party libraries such as `include/qpp`, `include/qasmtools`, and `include/Eigen`.

3 HHL Implementation

The HHL part loads a small complex linear system from `data/linear_system.txt`. The input handler checks whether the matrix is Hermitian. If it is not Hermitian, the code embeds the system into a larger Hermitian block matrix of the form

$$C = \begin{bmatrix} 0 & A \\ A^\dagger & 0 \end{bmatrix}.$$

The matrix and vector are then scaled, and the right-hand-side vector is normalized so that it can be represented as a quantum state.

The solver simulates the following HHL stages:

1. Prepare the state containing the clock register, target register $|b\rangle$, and ancilla.
2. Apply quantum phase estimation using controlled powers of $\exp(-iAt_0)$.
3. Apply a conditional ancilla rotation based on the estimated eigenvalue.

4. Apply inverse phase estimation to uncompute the clock register.
5. Measure the ancilla and return the postselected target-register state.

For the bundled two-dimensional example, the HHL target compiles and produces a reasonable normalized solution state. This confirms that the main simulation pipeline works for a very small demonstration case.

3.1 HHL Limitations

The HHL implementation is a useful educational simulation, but it is not a general or scalable HHL solver.

- Quantum++ stores the full state vector on a classical computer. The memory cost grows exponentially with the number of simulated qubits. Hence, the current code is limited to small systems.
- The code uses dense matrix operations, including dense Hamiltonian simulation of $U = e^{-iAt_0}$.

4 Nozzle-Flow and QAEA Implementation

The nozzle part is based on the paper’s reduction of the Navier–Stokes problem to a nonlinear ODE problem, followed by quantum mean estimation inside the ODE update. In one spatial dimension, the compressible Navier–Stokes equations can be written in conservation form as

$$\frac{\partial U}{\partial t} + \frac{\partial}{\partial x} F \left[U, \frac{\partial U}{\partial x} \right] = J[U],$$

where $U = (U_1, U_2, U_3)^T$ contains the conserved flow variables, F contains the flux terms, and J contains source terms. For the inviscid quasi-one-dimensional nozzle case considered in my implementation, the viscosity and thermal-conductivity contributions are omitted, so the flux function is simpler but the same algorithmic structure is used.

Physically, the test problem is flow through a convergent-divergent de Laval nozzle. The cross-sectional area decreases from the inlet to the throat and then increases toward the outlet. In the no-shock case, the flow accelerates from subsonic speed to sonic speed at the throat and becomes supersonic in the divergent section. In the shock case, a normal shock can appear in the divergent section and the flow drops from supersonic to subsonic speed across the shock. My current default run focuses on the no-shock configuration.

The default geometry matches the paper’s nozzle area:

$$A(x) = 1 + 2.2(x - 1.5)^2,$$

on the interval $[0, 3]$, with gas parameter $\gamma = 1.4$.

The conservative variables used in the code are

$$U_1 = \rho A, \quad U_2 = \rho A v, \quad U_3 = \rho A \left(\frac{T}{\gamma - 1} + \frac{\gamma}{2} v^2 \right).$$

After replacing the spatial continuum with a uniform grid

$$x_0, x_1, \dots, x_{m-1}, \quad \Delta x = x_{j+1} - x_j,$$

the spatial derivative is approximated by finite differences. In the current code, the interior derivative uses a central difference, schematically

$$\left. \frac{\partial F}{\partial x} \right|_{x_j} \approx \frac{F(U_{j+1}) - F(U_{j-1}))}{2\Delta x}.$$

Moving the spatial derivative and source terms to the right-hand side gives a finite-dimensional ODE system

$$\frac{dU}{dt} = f(U).$$

Here U represents the collection of conservative variables over all grid points, and $f(U)$ is the driver function implemented by the quasi-1D nozzle driver.

The time interval $[0, T]$ is divided into outer intervals

$$I_i = [t_i, t_{i+1}], \quad h = \frac{T}{n}.$$

Each outer interval is further divided into

$$N_k = n^{k-1}$$

sub-subintervals

$$I_i^l = [t_i^l, t_i^{l+1}], \quad \bar{h} = \frac{h}{N_k} = \frac{T}{n^k}.$$

At the beginning of an outer interval, $y_i(j)$ approximates the exact solution $U(j, t_i)$. Inside each sub-subinterval, the solution is represented locally by a truncated Taylor polynomial $l_{i,l}(j, u)$, where $u \in [0, 1]$ parameterizes the sub-subinterval. In the current code, the implemented case is first order:

$$l_{i,l}(u) \approx U_{\text{start}} + u \bar{h} f(U_{\text{start}}).$$

This corresponds to the relation $dU/dt = f(U)$. For a higher Taylor order, additional time derivatives of U would be needed.

The paper's update formula can be viewed as advancing the outer-interval solution by averaging the driver over all sub-subintervals:

$$y_{i+1}(j) = y_i(j) + N_k \sum_{l=0}^{N_k-1} \frac{\bar{h}}{N_k} \int_0^1 f(l_{i,l}(j, u)) du.$$

Equivalently, this is a sum of mean derivative contributions from the sub-subintervals. In my code, for each sub-subinterval the stepper samples $f(l_{i,l}(j, u))$ at knot values of u , normalizes

these samples to $[0, 1]$, and asks a mean-estimation backend to estimate their average. The state is then updated by

$$U_{\text{end}} \approx U_{\text{start}} + \bar{h} \bar{f},$$

where $\bar{f}$ is the estimated mean of the driver over the sampled knots.

The mean-estimation layer allows the nozzle time-stepper to use either:

- a classical backend that computes the exact discrete mean of sampled values, or
- a Quantum++ backend that simulates a QAEA-style amplitude-estimation procedure.

The QAEA-style backend pads sample data to a power-of-two size and encodes each normalized sample $g_j \in [0, 1]$ into an ancilla amplitude. Conceptually, the value-loading step prepares amplitudes of the form

$$\sqrt{1 - g_j} |0\rangle + \sqrt{g_j} |1\rangle$$

conditioned on the index state $|j\rangle$. The backend then builds a Grover iterate, applies phase estimation, and decodes the measured phase into an estimated mean. This QAEA-style mean estimate corresponds to the quantum part of the paper’s ODE update; the spatial discretization, Taylor construction, and boundary-condition handling remain classical in the present simulation.

5 Relation to the Reference Paper

The implementation matches several high-level components of the nozzle-flow paper: the de Laval nozzle geometry, the quasi-one-dimensional inviscid flow model, the reduction to an ODE system, and the use of quantum amplitude estimation as a mean-estimation subroutine.

However, the current code is not yet a full reproduction of the paper’s published numerical results. The comparison is summarized below.

Feature	Paper	Current implementation
Nozzle area $A(x)$	$1 + 2.2(x - 1.5)^2$	Implemented
Gas parameter	$\gamma = 1.4$	Implemented
No-shock case	Included	Partially implemented
Shock case with exit pressure	Included, $p_e = 0.6784$	Not fully reproduced
Taylor order	$r = 2$ in reported setup	Currently $r = 1$
Published-scale parameters	Larger simulation settings	Small demonstration settings
Validation against reported flow results	Included in paper	Still needed

6 Current Limitations

The main limitations are as follows.

1. **Quantum simulation cost.** The QAEA backend is simulated using Quantum++ state vectors, so it is expensive and scales poorly as the number of qubits grows.
2. **Simplified time integration.** The current local model uses first-order Taylor stepping. The reference paper uses a higher-order method in the reported simulation setup.
3. **Small demonstration parameters.** The current configuration is intentionally small so that the code can be compiled and tested locally. It does not yet reproduce the full paper-scale experiment.
4. **Shock case not fully reproduced.** The code includes a subsonic outlet option and an exit pressure parameter, but the default run is the supersonic outflow no-shock configuration.
5. **Limited quantitative validation.** The classical and QAEA backends can be compared structurally, but the code should report numerical errors, runtime, and agreement against classical or published reference solutions.
6. **Oracle model is explicit.** The QAEA backend explicitly loads all sample values through simulated controlled rotations. This is reasonable for a small simulator but does not model an efficient large-scale QRAM or oracle implementation.

7 Possible Next Steps

I would like advice on which direction is most valuable for further development. Possible next steps include:

- implementing the second-order Taylor method used in the paper;
- reproducing the paper’s no-shock and shock-case parameters more faithfully;
- adding quantitative validation against the classical backend and published flow profiles;
- improving QAEA diagnostics, including estimated error, success probability, and runtime;
- simplifying or refactoring the QAEA backend for clarity;
- deciding whether the project goal should remain simulation-based or shift toward a more theoretical oracle-complexity presentation.

8 Questions for Discussion

The main questions I hope to discuss are:

1. Should the next priority be physical fidelity to the nozzle-flow paper, or deeper validation of the QAEA subroutine?
2. Would reproducing the shock case be a valuable milestone?
3. What level of numerical comparison would be expected for a convincing project result?